

HOWTO — object-oriented C, and object-oriented assembly

A practical guide to doing OOP **without a C++ compiler**: in plain C, and one level down in assembly. Both are the *same* idea — an **object is data plus a table of code addresses (a vtable)**, and a **virtual call is an indirect call** through that table with a `this` pointer in a register. C++ compilers emit exactly this; you can write it by hand.

General first, then the **65816 / llvm-mos** specifics — including the *measured* assembly llvm-mos emits for a virtual call (this repo's bar: measure, don't assume). Written while building the SNES demo; it's the natural way to structure the rendering library (see [the rendering handoff](#)). That library — `snesgfx`, an OOP-in-C SNES renderer that is the first real, verified consumer of these patterns — is specified in [its plan](#); the *measured* dispatch numbers from building it land back in §4–§5 here (assertion → measurement).

1. The one idea

```
object  → [ vtable* | field | field | ... ]      (data; first word points at a vtable)
          |
          ▼
vtable  → [ &method0 | &method1 | ... ]          (a table of code addresses)

virtual call = load object[0] (vtable*) → load vtable[k] (method addr) → call it(self, ...)
```

Everything below is a way to spell that, with more or less sugar. Two axes decide how much machinery you need:

- **Static dispatch** — the concrete type is known at the call site, so you call the function *directly*. No vtable, no indirection. Cheapest. Use it whenever you can.
 - **Dynamic dispatch** — the call site only has a base type / interface; the actual method depends on the runtime object. That's what needs a vtable + an indirect call.
-

2. Object-oriented C

The four OOP pillars, done by hand:

Encapsulation

Methods are ordinary functions taking an explicit `self`. Hide data with **opaque pointers**: forward-declare the struct in the header, define it only in the `.c`. Callers can hold a `Foo *` but can't see or touch the fields.

```
/* foo.h */  typedef struct Foo Foo;   Foo *foo_new(void);   int foo_get(const Foo *);
/* foo.c */  struct Foo { int x; };   /* definition is private to this TU */
```

A clean object interface is *only* constructors + methods — no bare (receiver-less) procedures. Every public function takes the object it acts on as its first parameter (`Type_verb(Type *self, ...)`); a global `do_thing()` that pokes shared state behind the caller's back is the procedural shape OOP is meant to replace. The payoff is that an **invariant wrapped in a constructor cannot be bypassed**: e.g. a driver whose hardware bring-up sequence (reset → configure → enable, *in that order*) lives in `device_open(Device *)` and whose only mutators are methods on `Device` makes the dangerous ordering un-expressible by the caller — they can't forget the reset or enable too early, because the bare register pokes aren't in the interface. (This is exactly how the `snesgfx` rendering library hides the SNES boot bracket and the v-blank access-window rule: a `Display` constructor + methods, no `REG_*` in client code.)

Inheritance — "base struct first"

Make the base the **first member** of the derived struct. C guarantees a pointer to a struct equals a pointer to its first member, so the upcast `(Base*)derived` is valid and free. (This is precisely how C++ lays out single inheritance.)

```
typedef struct { Shape base; uint16_t w, h; } Rect;    /* Rect "is-a" Shape */
Rect r;  Shape *s = (Shape*)&r;                      /* upcast: legal, zero-cost */
```

Downcast is a plain cast back (`(Rect*)s`) — *you* are responsible for it being the right type (or carry a type tag / compare the vtable pointer).

Polymorphism — vtables

A vtable is a `struct` of function pointers; each object points at one. The "virtual call" dereferences through it. Full canonical pattern:

[illegible]

```

    uint16_t (*area)(const Shape *self);
    void      (*draw)(const Shape *self);
} ShapeVT;
struct Shape { const ShapeVT *vt; }; /* base object: vtable pointer FIRST */

/* dispatchers – the "virtual call" */
static inline uint16_t shape_area(const Shape *s){ return s->vt->area(s); }
static inline void      shape_draw(const Shape *s){ s->vt->draw(s); }

/* a concrete type: embeds Shape as member 0, supplies its own methods + vtable */
typedef struct { Shape base; uint16_t w, h; } Rect;
static uint16_t rect_area(const Shape *s){ const Rect *r=(const Rect*)s; return (uint16_t)(r->w*r->h); }
static void      rect_draw(const Shape *s){ /* ... */ }
static const ShapeVT RECT_VT = { rect_area, rect_draw }; /* one vtable per class */
static void rect_init(Rect *r, uint16_t w, uint16_t h){ r->base.vt=&RECT_VT; r->w=w; r->h=h; }

/* another concrete type – same interface, different behaviour */
typedef struct { Shape base; uint16_t r; } Circle;
static uint16_t circle_area(const Shape *s){ const Circle *c=(const Circle*)s; return (uint16_t)(3*c->r*c->r); }
static const ShapeVT CIRCLE_VT = { circle_area, /*draw=*/0 };

```

Now `shape_area(s)` runs `rect_area` or `circle_area` depending on the object — real runtime polymorphism, no C++.

Construction / destruction

Just `init` / `deinit` (or `new` / `free`) functions. The constructor's job includes wiring the vtable pointer (`r->base.vt = &RECT_VT`). A "virtual destructor" is a `void (*dtor)(Shape*)` slot in the vtable.

This is everywhere

The pattern isn't exotic — it's how large C codebases do OOP:

- **GLib/GTK's GObject** — vtables (`*Class` structs), single inheritance by first-member, `G_DEFINE_TYPE` .
- **The Linux kernel** — `struct file_operations` , `struct net_device_ops` , ... are vtables; `container_of()` is the downcast.
- **CPython** — `PyTypeObject` is a big vtable; every `PyObject` starts with `ob_type` .
- **C++ itself** lowers single-inheritance virtual calls to exactly this.

3. Object-oriented assembly

Strip the syntax and it's the same memory shapes:

- An **object** is a block you lay out yourself; field offsets are constants.
- A **vtable** is a label followed by code-address entries (`.addr` / `.word`).
- A **virtual call** is: load the vtable pointer from `self+0` , load the method address at a fixed offset, **indirect-call** it with `self` in the argument register.

On a CPU with a cheap indirect call this is short. E.g. x86 (Itanium/SysV-ish, `this` in the first arg / `rdi`):

```
mov    rax, [rdi]          ; rax = self->vtable
call   [rax + 8*K]         ; call vtable[K], self already in rdi
```

The interesting cases are 8/16-bit CPUs **without** a direct "call through a register/pointer" instruction — like the 6502/65816 — where you must build the indirect call. There are two distinct situations, and they want different idioms:

(a) Static, per-class *selector* tables — cheap

If you dispatch by a **message id / selector** and the table lives at a **link-time address** (one table per class, not per object), the 65C02/65816 has a purpose-built instruction:

op	mnemonic	does
FC	JSR (abs,X)	jump-to-subroutine through a table indexed by X
7C	JMP (abs,X)	same, as a jump

```
; X = selector * 2 (word entries); VT is a fixed table of method addresses
asl    a                ; selector -> *2 (if it was in A)
tax
jsr    (VT,x)           ; calls VT[selector]; the method returns with RTS
; ...
VT:    .addr m_draw, m_area, m_update      ; the class's method table
```

One instruction, no per-object storage. This is the sweet spot on 6502-family hardware.

(b) Per-object vtables — flexible, a touch more work

If the vtable pointer lives **in the object** (true per-instance polymorphism), the address is a *runtime* value, so `JSR (abs,X)` (which bakes the table address into the opcode) can't be used directly. Two portable idioms:

1. **Load the method pointer into a zero-page vector, then** `JMP (vector)` (`6C`). For a *call* (not a tail), `JSR` a tiny thunk that does the `JMP (vector)` — the target's `RTS` then returns to your caller. (This is exactly what `llvm-mos` does — see §4.)
2. **The `RTS` trick** — push `target-1` (hi then lo) and execute `RTS` ; the CPU "returns" to `target` . Pure 6502, no extra ZP.

Far (24-bit) methods that live in another bank use `JML [dp]` (`DC` , indirect-long) or the `RTL` equivalent of the trick.

65816 indirection opcode reference: `6C` `JMP (a)` , `7C` `JMP (a,X)` , `FC` `JSR (a,X)` , `DC` `JML [a]` , `5C` `JML (direct long)` , `22` `JSL (direct long subroutine)`. There is no `JSL (...)` indirect-long-subroutine — build it from a push + `RTL` or a thunk.

4. How `llvm-mos` lowers a C virtual call to 65816 (measured)

You don't have to hand-roll any of this if you write OO C — `llvm-mos` (a real LLVM backend; the SDK even compiles C++) lowers the indirect call for you. Here is the *actual* emitted code for the canonical virtual call, `uint16_t poly_area(const Shape *s){ return s->vt->area(s); }` , at `-mcpu=mosw65816 -Os` :

```
poly_area:
    lda    (__rc2)          ; s is the 'this' ptr, passed in the ZP pair __rc2/__rc3
    sta    __rc4
    ldy    #1
    lda    (__rc2),y        ; load s->vt (object offset 0) -> __rc4/__rc5
    sta    __rc5
    lda    (__rc4)
    sta    __rc18
    lda    (__rc4),y        ; load vt->area (vtable offset 0) -> __rc18/__rc19
    sta    __rc19
    jmp    __call_indir     ; tail-call the indirect-call thunk
```

and the thunk `__call_indir` is a *one-instruction* trampoline:

```
__call_indir:
    jmp     (__rc18)        ; 6C 12 00 - indirect jump through __rc18 ($0012)
```

So `llvm-mos`'s recipe is idiom (b.1) above: **load the target into the ZP vector `__rc18` , then `JMP (__rc18)`** . The `this` pointer travels in the imaginary-register pair `__rc2/__rc3` (the calling convention's first pointer arg). Because `poly_area` *tail-jumps* to `__call_indir` , the method's own `RTS` returns straight to `poly_area` 's caller. The same thunk serves a non-tail call too: `JSR __call_indir` → `JMP (__rc18)` → method → `RTS` lands after the `JSR` .

Takeaways for the rendering library:

- A C vtable "just works" on the 65816 — but every virtual call is ~8 ZP loads/stores + a `JMP (vector)`, plus the callee can't be inlined. **Far more expensive than a direct call.**
- Function pointers in general (callbacks, jump tables) lower through this same `__call_indir` path.

5. Costs & when to use which (8/16-bit reality)

Cheapest → dearest:

1. **Direct call** (static dispatch) — `JSR target`. Inlinable, no indirection. Default to this.
2. **Static selector table** (`JSR (abs,X)`) — one class, dispatch by id; no per-object RAM, one indexed indirect jump. Great for state machines, command/opcode dispatch, a single widget kind with many message types.
3. **Per-object vtable** — full polymorphism across heterogeneous objects, at the cost of a pointer per object (2 bytes near / 3 far) **and** the `__call_indir` sequence per call.

On a 3.58 MHz CPU with no hardware indirect-call, reserve per-object vttables for genuinely polymorphic collections (e.g. "a list of drawables of different kinds"). If every element is the same type, use static dispatch. If you're dispatching one object by many message kinds, use a selector table. Don't pay for dynamic dispatch you don't use — the C compiler will **devirtualize** a call only when it can prove the concrete type, which across translation units it usually can't.

Budget your dispatch coarsely — the load-bearing rule for a real-time renderer. A virtual call costs ~8 ZP loads/stores + a `JMP (vector)` and **does not inline** (§4). So put the vtable seam where the type is genuinely unknown and call it *rarely*: in a renderer, **one virtual call per drawable per frame** (`drawable->emit(self, queue)` — "upload yourself"), never inside the per-tile/per-pixel inner loop. One indirect call decides *which kind of thing* runs; the hot loop after that is type-known and free (direct/inlined byte+DMA). A per-tile virtual call, by contrast, would let the ~8-ZP-op sequence dominate the frame. The cost of polymorphism is then `N_objects` indirect calls per frame — negligible — instead of `N_pixels`.

Memory-model notes (this target):

- Vtables and near method pointers are **16-bit** (bank `$00`); a `const ShapeVT` lives in ROM/ `.rodata`. Keep the vtable and all methods in the same bank and dispatch stays 16-bit.
- Methods in another bank ⇒ method pointers are **24-bit far**; the indirect call must use `JML [dp]`, and a far function pointer is a 32-bit value (`+mos-a16 -only`, like any far pointer — see §4 of [the rendering handoff](#)).

- The vtable pointer costs 2 bytes of RAM per *object* (it's a field). The vtable *itself* is shared (one per class, in ROM) — don't duplicate it per instance.

6. Recipes

Portable OO-C interface (drop-in)

The §2 `Shape / Rect / Circle` pattern is the whole recipe: a `*VT` struct (interface), a base with the vtable pointer first, concrete types embedding the base as member 0, one `const vtable` per class, an `init` that wires it. Call through `static inline` dispatchers so the call sites read like methods. A heterogeneous container is then just `Shape *items[N];` and a loop calling `shape_draw(items[i])`.

Heterogeneous scene of drawables (the rendering library's actual shape)

This is the canonical case where a per-object vtable *earns* its cost (§5): a frame is a list of **different kinds** of thing — a Mode 7 layer, a sprite set, a tiled BG — each of which knows how to upload itself, but the render loop doesn't know (or want to know) which is which. `Drawable` is the interface; every concrete layer embeds it as member 0; the scene is `Drawable *items[N]`:

```
typedef struct Drawable Drawable;
typedef struct { void (*emit)(Drawable *self, UploadQueue *q); } DrawableVT; /* coarse: "upload your struct Drawable { const DrawableVT *vt; };
```

```
typedef struct { Drawable *items[SCENE_MAX]; uint8_t n; } Scene;
static inline void scene_emit(Scene *s, UploadQueue *q) {
    for (uint8_t i = 0; i < s->n; i++)
        s->items[i]->vt->emit(s->items[i], q);    /* ONE indirect call per drawable per frame */
}

/* a concrete drawable embeds Drawable first; its emit() inner loop is monomorphic & inlined */
typedef struct { Drawable base; const M7_FAR uint8_t *src; /* ... */ } Mode7Layer;
static void mode7_emit(Drawable *self, UploadQueue *q) { Mode7Layer *m=(Mode7Layer*)self; /* tight D
static const DrawableVT MODE7_VT = { mode7_emit };
```

The vtable seam is exactly where the **dispatch-budget rule** (§5) wants it: `scene_emit` pays `N_drawables` indirect calls per frame (a Mode 7 layer + a sprite HUD = 2), and the expensive upload happens in each `emit`'s *type-known* inner loop. A single-type scene skips all of this and calls the concrete `mode7_emit` directly. Worked, differentially-verified end to end (CRC `0x204F` on both emulators) by [snesgfx](#).

Hand-asm selector dispatch (when you skip C for the hot path)

```
; dispatch(obj_ptr in __rc2/3, selector in A) for a single class with a fixed method table
    asl    a                ; selector*2 (word table)
    tax
    jmp    (CLASS_VT,x)     ; tail-dispatch; method returns to our caller via RTS
CLASS_VT:
    .addr  on_reset, on_tick, on_draw      ; selectors 0,1,2
```

Verify it like everything else in this repo

Tie behaviour to a number: have each method write a sentinel/accumulate a CRC, then assert it on both emulators (`dev/jgxcheck.cpp` + a MAME Lua), exactly as the Mandelbrot demo does. A polymorphic dispatch that "looks right" isn't verified until the bytes agree.

7. The addressing modes that make this cheap — a note on the design

Everything above leans on the same handful of addressing modes, and it's worth saying why they're there. The 6502 is famously **register-poor** — one 8-bit accumulator, two 8-bit index registers, an 8-bit stack pointer nailed to page 1 — and the usual conclusion is that it's a poor target for compiled, high-level, or object-oriented code. That misreads the design: the richness lives in the **addressing modes**, not the register file, and those modes map almost one-to-one onto the shapes this document is built from (pointers, arrays, structs, vtables, call frames).

addressing mode	what it does	the HLL / OOP construct it serves
zero page <code>dp</code>	one-byte address into page 0	the compiler's <i>register file</i> — llvm-mos parks its imaginary registers (<code>__rc*</code>) here; this answers "too few registers"
<code>(dp),Y</code> — indirect indexed	<code>*(ptr + Y)</code> , pointer held in ZP	<code>ptr->field</code> in one instruction (member offset in Y); also <code>*ptr</code> and byte-array <code>p[i]</code> — the workhorse mode for compiled code
<code>(dp,X)</code> — indexed indirect	pick the X-th pointer from a ZP table, then deref	arrays of pointers, callback tables

addressing mode	what it does	the HLL / OOP construct it serves
<code>abs,X / abs,Y</code>	base + index	indexing a global / <code>static</code> array
<code>JSR (abs,X)</code> (65C816)	call through an X-indexed table of code addresses	a hardware vtable / selector dispatch — §3(a): one instruction, no per-object storage
<code>sr,S / (sr,S),Y</code> (65816)	stack-pointer-relative	C call frames — locals and arguments, recursion, re-entrancy
relocatable Direct Page <code>D</code> (65816)	a movable "zero page"	a per-task / per-call register window (<code>tsc</code> ; <code>phd</code> ; <code>tcd</code>) — re-entrant code, multitasking
<code>[dp] / [dp],Y</code> (65816)	24-bit long indirect	far pointers and far methods across banks (§3, §5)

Two things stand out. First, **zero page is really a 256-byte register file** — which is exactly how a modern optimizer treats it, so the register-poverty critique is answered by the memory model rather than the register count. Second, the indexed-indirect family — `JSR (abs,X)` in particular — puts method dispatch *in the instruction set*: a vtable call, in one instruction, decades before vtables were fashionable.

And the sharpest case is **struct-field access**: a struct pointer in zero page, the member's offset in Y, and `ptr->field` is a single `lda (ptr),Y` — exactly the `s->vt` load measured in §4. Arrays of multi-byte elements need the index scaled first, so for anything wider than a byte `ptr->field` is the cleaner fit than `p[i]`.

The original 6502 already shipped 13 addressing modes in a \$25 1975 part, when rival CPUs cost an order of magnitude more — richer addressing than its contemporaries, at a fraction of the price. The 65816 then added precisely the modes that compiled and re-entrant code had been missing — stack-relative frames, a relocatable direct page, and 24-bit long-indirect pointers — which is why the WDC816CC / ORCA-C frame ABI (see [the prior-art note](#)) and llvm-mos's own lowering land where they do. In hindsight the '816 reads like an architecture waiting for the optimizing compiler its addressing modes were designed to reward.

8. References

- This repo: the measured lowering above (`build/llvm-mos-install/bin/mos-clang --target=mos -mcpu=mosw65816 -Os -S` on a vtable call); the rendering handoff [docs/handoffs/2026-06-24-snes-](#)

[graphics-rendering.md](#) .

- **GObject** reference (GLib) — the canonical large-scale OO-C system.
- **"Object-Oriented Programming With ANSI-C"** (Axel-Tobias Schreiner) — the classic free book on exactly these techniques.
- The Linux kernel's `container_of()` + `*_ops` vtables; CPython's `PyTypeObject` .
- **fullsnes** / **anomie** for the 65816 indirection opcodes (`JSR (a,X)` etc.).